

UFMA – Ciência da Computação Estrutura de Dados I

Obs.: Não pode usar vetores auxiliares.

$$i \neq col + j - 1 \Rightarrow i \neq col + j$$

1. Faça um algoritmo que recebe duas matrizes de inteiros **A** e **B** com **n** linhas e **m** colunas, armazenadas nos vetores **va** e **vb**. Calcule e armazene em **vc** o resultado de **A menos a transposta de B**. **Atencao:** Para que a operação seja válida, a transposta de B deve ter as mesmas dimensões de A.

int SubtraiAPorTransposta(int n, int m, int *va, int *vb, int *vc);

2. Faça um algoritmo que recebe uma string **s** com uma frase (palavras separadas por espaços). Usando o TAD Pilha, gere uma nova frase em que as palavras nas posições ímpares (1a, 3a, 5a, ...) tem suas letras invertidas; as palavras nas posições pares permanecem inalteradas. A ordem das palavras na frase deve ser mantida.

Exemplo: *faz a copia e/ou insere na pilha*

"ESTE EXERCICIO Esta MUITO FACIL"

=> "ETSE EXERCICIO atSE MUITO LICAF"

(pos) 1 "ESTE" -> invertida, pos 2 "EXERCICIO" -> inalterada, ...)

char *InvertePalavrasNasPosImpares(char *s, int n);

3. Faça um algoritmo que move o elemento que está na posição **N** a partir do topo para o fundo da pilha, mantendo a ordem relativa de todos os demais elementos. Retorne 1 se bem-sucedido, ou 0 se a pilha tiver menos de **N** elementos.

typedef struct {

void **item; int top; int maxItens;

} Stack;

int MoveNEsimoParaFundo(Stack *s, int n);

Exemplo com **N = 2** (topo a esquerda):

Antes: [3 7 1 9 7 2] (topo=3, fundo=2)

Move o elemento na posicao 2 (valor 7) para o fundo:

Depois: [3 1 9 7 2 7]

Obs.: Manipule diretamente a estrutura interna da pilha. Não pode usar funções do TAD (push, pop) nem vetores auxiliares.

4 - Receba uma fila circular armazenada em vetor e um inteiro **k**. Mova o **k**-esimo elemento da fila contando a partir da frente para a ultima posição da fila, preservando a ordem relativa de todos os demais elementos. Retorne 1 se bem-sucedido, ou 0 se a fila tiver menos de **k** elementos.

typedef struct {

void **item; int front; int rear; int nElms; int

maxItens;

} Queue;

Resolva manipulando diretamente a estrutura interna da fila circular, sem usar as funções do TAD (qEnqueue, qDequeue) e sem usar vetores auxiliares.

Exemplo com **k = 2**:

Antes: [A B C D E] (frente = A, fim = E)

Move B (2o elemento) para o final:

Depois: [A C D E B]

int MoveKEsimoParaUltimo(Queue *q, int k);

Obs.: Não pode usar funcoes do TAD nem vetores auxiliares.

L > cria outro vetor

n = 3



nao conta indice

k = 3

